

```
#include <iostream>
```

```
#include <fstream>
```

```
#include <string>
```

```
using namespace std;
```

```
bool isLetter(char c){
```

```
    return (c >= 'A' && c <= 'Z') || (c >= 'a' && c <= 'z');
```

```
}
```

```
bool isDigit(char c){
```

```
    return (c >= '0' && c <= '9');
```

```
}
```

```
bool isKeyword(string s){
```

```
    return s=="int" || s=="float" || s=="double" || s=="char" ||
```

```
        s=="void" || s=="if" || s=="else" || s=="while" || s=="return";
```

```
}
```

```
bool isIdentifier(string s){
```

```
    if(s.length() == 0){
```

```
        return false;
```

```
}
```

```
    if(!(isLetter(s[0]) || s[0] == '_')){
```

```
        return false;
```

```
}
```

```
    for(int i=1; i<(int)s.length(); i++){
```

```

        if(!(isLetter(s[i]) || isDigit(s[i]) || s[i] == '_')){
            return false;
        }
    }

    return true;
}

```

```

bool isOperator(char c){
    return c=='+' || c=='-' || c=='*' || c=='/' ||
        c=='=' || c=='<' || c=='>';
}

```

```

bool isDelimiter(char c){
    return c=='(' || c==')' || c=='{' || c=='}' ||
        c=='[' || c==']' || c==',' || c==';';
}

```

```

int main()
{
    ofstream fout("input.txt");

    int n;
    cout << "How many lines: ";
    cin >> n;

    string line;
    getline(cin, line);
}

```

```
for(int i=0; i<n; i++){  
    cout << "Enter line " << i+1 << ": ";  
    getline(cin, line);  
    fout << line << endl;  
}
```

```
fout.close();
```

```
ifstream fin("input.txt");  
ofstream tokens("tokens.txt");  
ofstream symbol("symbol_table.txt");
```

```
string ids[1000];  
int firstLine[1000];  
int idCount = 0;
```

```
int keywordCount = 0;  
int identifierCount = 0;  
int integerCount = 0;  
int operatorCount = 0;  
int delimiterCount = 0;
```

```
int lineNo = 0;
```

```
while(getline(fin, line)){  
    lineNo++;  
    int i = 0;  
  
    while(i < (int)line.length()){
```

```
if(line[i] == ' ' || line[i] == '\t'){
```

```
    i++;
```

```
}
```

```
else if(isLetter(line[i]) || line[i] == '_'){
```

```
    string token = "";
```

```
    while(i < (int)line.length() &&
```

```
        (isLetter(line[i]) || isDigit(line[i]) || line[i] == '_')){
```

```
        token += line[i];
```

```
        i++;
```

```
}
```

```
if(isKeyword(token)){
```

```
    tokens << "Line " << lineNo << ": " << token << " -> Keyword" << endl;
```

```
    keywordCount++;
```

```
}
```

```
else if(isIdentifier(token)){
```

```
    tokens << "Line " << lineNo << ": " << token << " -> Identifier" << endl;
```

```
    identifierCount++;
```

```
bool found = false;
```

```
for(int j=0; j<idCount; j++){
```

```
    if(ids[j] == token){
```

```
        found = true;
```

```
        break;
```

```
    }
```

```
}
```

```

        if(!found){
            ids[idCount] = token;
            firstLine[idCount] = lineNo;
            idCount++;
        }
    }
}

else if(isDigit(line[i])){
    string token = "";

    while(i < (int)line.length() && isDigit(line[i])){
        token += line[i];
        i++;
    }

    tokens << "Line " << lineNo << ": " << token << " -> Integer Constant" << endl;
    integerCount++;
}

else if(isOperator(line[i])){
    tokens << "Line " << lineNo << ": " << line[i] << " -> Operator" << endl;
    operatorCount++;
    i++;
}

else if(isDelimiter(line[i])){
    tokens << "Line " << lineNo << ": " << line[i] << " -> Delimiter" << endl;

```

```
        delimiterCount++;  
        i++;  
    }
```

```
    else{  
        i++;  
    }  
}  
}
```

```
tokens << endl;  
tokens << "Summary:" << endl;  
tokens << "Keywords = " << keywordCount << endl;  
tokens << "Identifiers = " << identifierCount << endl;  
tokens << "Integer Constants = " << integerCount << endl;  
tokens << "Operators = " << operatorCount << endl;  
tokens << "Delimiters = " << delimiterCount << endl;
```

```
for(int i=0; i<idCount; i++){  
    symbol << i+1 << " " << ids[i] << " " << firstLine[i] << endl;  
}
```

```
fin.close();  
tokens.close();  
symbol.close();
```

```
return 0;  
}
```

Output:

```
int sum = a1 + 25;
float x;
if(x>0) return x;
```

```
Line 1: int -> Keyword
Line 1: sum -> Identifier
Line 1: = -> Operator
Line 1: a1 -> Identifier
Line 1: + -> Operator
Line 1: 25 -> Integer Constant
Line 1: ; -> Delimiter
Line 2: float -> Keyword
Line 2: x -> Identifier
Line 2: ; -> Delimiter
Line 3: if -> Keyword
Line 3: ( -> Delimiter
Line 3: x -> Identifier
Line 3: > -> Operator
Line 3: 0 -> Integer Constant
Line 3: ) -> Delimiter
Line 3: return -> Keyword
Line 3: x -> Identifier
Line 3: ; -> Delimiter

Summary:
Keywords = 4
Identifiers = 5
Integer Constants = 2
Operators = 3
Delimiters = 5
```

```
1 sum 1
2 a1 1
3 x 2
```